READVAULT

STORY TIME

Feature Specification & Implementation Brief

A child-friendly storybook creator inside the existing ReadVault application

Implementation target: existing Next.js starter • Initial budget: ■0 • Local-first MVP

Prepared as a handoff document for an AI coding assistant (DeepSeek or Codex)

1. Purpose and Product Vision

Add a new feature called **Story Time** to ReadVault. ReadVault is a personal digital library and PDF reading application. Story Time should let a parent create, manage, and read illustrated children's stories, including original stories and age-appropriate movie-inspired summaries or retellings. The experience should be warm, simple, safe, and easy for a child to use with adult guidance.

This document is an implementation brief for an AI coding assistant. The assistant must inspect the existing ReadVault repository first, then implement the feature incrementally without replacing the existing application or breaking its current PDF-library functionality.

2. Key Product Decisions

- **Support both story modes:** (1) original stories created from scratch and (2) movie-inspired summaries or high-level retellings written in original wording.
- **Support both illustration workflows:** (1) original illustrations added to story pages and (2) user-uploaded drawings or pictures.
- **Keep the first version local-first:** use the existing SQLite database and local filesystem patterns already present in ReadVault.
- **Keep the initial budget at ■0:** do not require paid AI APIs, paid image APIs, paid hosting, or paid subscriptions for the MVP.
- **Make the feature optional and modular:** Story Time should be a separate section and must not interfere with PDF books.

3. Copyright and Intellectual Property Guardrails

Disney and other movie franchises are copyrighted. Story Time may allow a user to enter a movie title and create a brief, original-language summary or a new story inspired by broad themes such as friendship, courage, family, discovery, or kindness. Do not reproduce movie scripts, dialogue, scene-by-scene transcripts, or complete copyrighted books. Do not market generated stories as official Disney stories.

- Do not generate or source official Disney artwork, movie stills, logos, or exact copyrighted character designs.
- Prefer original characters, settings, costumes, and visual designs. A general theme or mood may inspire an original story, but the output should not imitate a specific protected character.
- Allow user-uploaded artwork only when the user has the right to use it.
- Label movie-inspired content clearly as unofficial and independently written.
- If a future workflow needs licensed characters or official images, require appropriate permission or licensing.

4. Main User Experience

Add a new navigation item named **Story Time** to the existing ReadVault sidebar. It should open a dedicated story library.

Story Time home screen

- Page title: Story Time, with a friendly subtitle such as 'A little library of magical adventures'.
- Display saved stories as attractive book cards with cover image, title, short description, age range, and reading progress.
- Provide a clear 'Create a Story' action.
- Provide filters or tabs for All Stories, Original Stories, and Movie-Inspired Stories.
- Include helpful empty states when no stories exist.

Create story flow

1. Choose story type: Original Story or Movie-Inspired Story.
2. Enter title or movie title, story idea, age range, language, and approximate length.
3. Choose an illustration approach: original illustration placeholder/uploaded image; do not require an external paid service.
4. Create the story as a sequence of editable pages.
5. Save as draft or publish to the child's Story Time library.

Story reader

- Display one illustrated page at a time, with readable large text and a page indicator.
- Previous/Next page controls and a simple progress indicator.
- Optional read-aloud control using browser SpeechSynthesis when supported.
- Allow parent/editor mode to edit page text, reorder pages, add or replace images, and delete pages.
- Persist reading progress so the story can be resumed.

5. Story Creation Requirements

Each story should support these fields:

- Title (required)
- Story type: original or movie-inspired
- Movie/source title (optional; used only as context for a brief summary or broad inspiration)
- Short description
- Target age range or reading level
- Language (English initially; design so more languages can be added)
- Cover image path (optional)
- Status: draft or published
- Created and updated timestamps

Each story page should support:

- Page order / page number
- Story text
- Optional illustration image path
- Alt text or short image description
- Created and updated timestamps

Provide reasonable defaults, but allow the parent to edit all generated or entered text before saving/publishing.

6. Illustration and Image Strategy

The first version must work without paid image-generation services. Implement the data model and UI so images can be uploaded locally and associated with individual story pages.

- Allow common image formats such as JPEG, PNG, and WebP with configurable size limits.
- Validate file type and size on the server; do not trust the extension alone.

- Use safe generated filenames and prevent path traversal.
- Store image files in a dedicated local directory separate from PDF uploads, for example data/story-images/.
- Show a placeholder illustration when a page has no image.
- Provide alt text for accessibility.
- Keep an abstraction point for a future image-generation provider, but do not integrate a paid API in the MVP.

Possible free image sources may be considered only when licensing and attribution requirements are checked. Do not scrape copyrighted movie artwork or copy image results from search engines.

7. Read-Aloud and Child-Friendly Accessibility

- Use the browser's built-in SpeechSynthesis API where available; this does not require a paid service.
- Provide Play, Pause, and Stop controls if supported by the browser.
- Handle unsupported browsers gracefully and keep text readable without audio.
- Use large, high-contrast typography and generous spacing.
- Use clear buttons with text labels or accessible names.
- Avoid flashing animations, autoplay audio, and distracting motion.
- Design responsive layouts for desktop, tablet, and phone.

8. Data Model Proposal

Inspect the current SQLite schema and database utilities before making changes. Adapt naming and migration conventions to the existing project rather than creating a competing database connection.

Suggested tables

- **stories**: id, title, slug (optional), story_type, source_title (nullable), description, age_range, language, cover_image_path, status, created_at, updated_at.
- **story_pages**: id, story_id, page_number, content, image_path (nullable), image_alt (nullable), created_at, updated_at.
- **story_progress**: id, story_id, current_page, last_read_at, updated_at. Enforce one progress record per user/story when authentication is introduced.

Suggested relationship: one story has many story_pages. A story may have zero or one cover image and each page may have zero or one illustration. Use foreign keys and cascading behavior carefully.

Example SQLite schema (adapt to project conventions)

```
CREATE TABLE stories ( id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT NOT NULL, story_type TEXT NOT NULL CHECK (story_type IN ('original', 'movie_inspired')), source_title TEXT, description TEXT, age_range TEXT, language TEXT DEFAULT 'en', cover_image_path TEXT, status TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'published')), created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP ); CREATE TABLE story_pages ( id INTEGER PRIMARY KEY AUTOINCREMENT, story_id INTEGER NOT NULL, page_number INTEGER NOT NULL, content TEXT NOT NULL, image_path TEXT, image_alt TEXT, created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (story_id) REFERENCES stories(id) ON DELETE CASCADE, UNIQUE (story_id, page_number) ); CREATE TABLE story_progress ( id INTEGER PRIMARY KEY AUTOINCREMENT, story_id INTEGER NOT NULL UNIQUE, current_page INTEGER NOT NULL DEFAULT 1, last_read_at TEXT, updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (story_id) REFERENCES stories(id) ON DELETE CASCADE );
```

9. Suggested Routes and Components

Follow the existing project structure and conventions. These are conceptual route suggestions, not a requirement to duplicate an existing pattern.

```
Routes: /story-time /story-time/new /story-time/[storyId] /story-time/[storyId]/edit Possible API endpoints: GET /api/stories POST /api/stories GET /api/stories/[storyId] PATCH /api/stories/[storyId] DELETE /api/stories/[storyId] POST /api/stories/[storyId]/pages PATCH
```

```
/api/stories/[storyId]/pages/[pageId] DELETE /api/stories/[storyId]/pages/[pageId] POST
/api/stories/[storyId]/images PUT /api/stories/[storyId]/progress Possible components: StoryTimeDashboard
StoryCard StoryCreationForm StoryEditor StoryPageEditor StoryReader StoryPage StoryProgress ImageUploader
ReadAloudControls
```

10. AI Generation: Phased Approach

Do not make paid AI generation a dependency. Build a working manual story editor first.

Phase A — no AI

- Parent writes or pastes original story text.
- Editor helps split the text into pages.
- Parent uploads illustrations or leaves placeholders.
- Stories can be saved, edited, read, and exported.

Phase B — optional local AI

- If the user chooses to install a local model, explore a local runtime such as Ollama.
- Check the user's hardware and model requirements before recommending a specific model.
- Make local generation optional; the rest of Story Time must work if no model is installed.
- Add age-appropriate output checks and parent review before publishing.

Phase C — optional external providers

Only add an external AI or image provider after explicit user choice, checking current pricing, terms, privacy, and API requirements. Never hard-code API keys in the frontend or commit secrets.

11. PDF Export

- Allow exporting a saved story as a PDF with a cover page and one story page per designed page.
- Include illustrations when present and preserve readable text.
- Use an appropriate page size and margins for phone-friendly or print-friendly reading.
- If PDF export is not feasible in the first implementation pass, leave a clear interface and document the deferred task rather than implementing a broken export.

12. Privacy and Child Safety

- Treat this as a parent-managed personal library, not a public social network.
- Do not add public profiles, comments, messaging, or public story sharing in the MVP.
- Do not collect unnecessary personal information about the child.
- Do not send story text, uploaded pictures, or identifying information to external services without explicit adult consent.
- Provide parent review before any generated content becomes visible in the child's library.
- Validate uploads and protect local file paths.

13. Implementation Plan for DeepSeek/Codex

1. **Inspect first:** inspect the full ReadVault repository, package.json, app routes, components, database setup, upload utilities, and styling conventions. Report the existing architecture before editing.
2. **Plan changes:** list files to add/change, proposed database migration, routes, and test plan. Do not overwrite or rebuild the existing app.

3. **Implement data layer:** add stories, story_pages, and story_progress using the existing SQLite connection and migration style.
4. **Implement CRUD:** create, list, view, edit, publish/draft, and delete stories and pages.
5. **Implement image uploads:** local story image storage, validation, safe filenames, and placeholders.
6. **Implement reader:** page-by-page reading, responsive layout, progress saving, and browser read-aloud.
7. **Implement export:** PDF export if it can be done reliably without paid services.
8. **Verify:** run lint, type-check, build, and tests; manually test create/edit/read/resume/delete flows.
9. **Report:** summarize changed files, migration steps, commands, tests run, known limitations, and any unresolved issues.

Mandatory coding constraints

- Preserve existing PDF library, reader, upload, and dashboard functionality.
- Use strict TypeScript and existing project conventions.
- Keep database access and file storage out of presentational components.
- Validate all inputs server-side; use parameterized SQL.
- Do not add paid services, API keys, or cloud dependencies for the MVP.
- Do not claim tests passed unless they were actually run.
- If the starter does not install or build, stop and report the exact error before expanding scope.

14. Definition of Done

- A Story Time navigation item is visible and works.
- A parent can create an original or movie-inspired story.
- A parent can add, edit, reorder, and delete story pages.
- A parent can upload an image for the cover or each page.
- Stories are saved in SQLite and images are stored locally.
- A story can be opened in a page-by-page reader.
- Reading progress is saved and restored.
- Read-aloud works where browser support exists and fails gracefully where it does not.
- Existing ReadVault PDF features still work.
- Lint, type-check, build, and available tests have been run and their results reported.

15. First Task to Give DeepSeek

Use the following instruction as the first implementation prompt:

```
Read the attached ReadVault Story Time Feature Specification. First inspect the existing repository thoroughly and report its current architecture, database setup, upload handling, route conventions, and any risks. Do not edit files yet. Then propose a small implementation plan that fits the existing codebase, including the exact files you expect to add or change, the SQLite migration, and the verification commands. Preserve all existing ReadVault functionality. Wait for approval before implementing.
```

After reviewing the inspection report, approve implementation in small milestones rather than asking the coding assistant to build everything in one large operation.